

AI AUTONOMOUS SWING TRADER

BLUEPRINT V1.0 — ARTEFATO MESTRE DO PROJETO

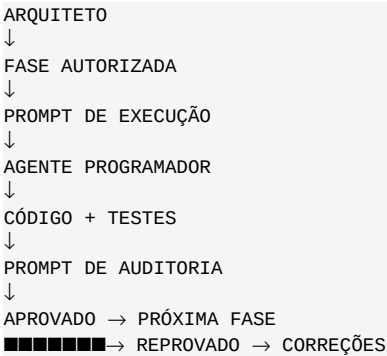
Documento consolidado contendo a visão conceitual, decisões da V1, arquitetura, governança e plano de implementação em 20 fases.

Estado: Blueprint conceitual e arquitetural consolidado; plano de implementação definido.

Ambiente: Docker em servidor Oracle, com operação exclusivamente virtual/sandbox na V1.

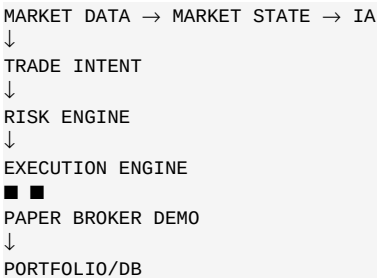
1. Propósito e Governança

Este documento é o artefato mestre do projeto. A arquitetura é definida pelo arquiteto; o agente programador executa somente a fase autorizada; uma segunda etapa audita a implementação antes da próxima fase.



2. Visão Conceitual V1

Plataforma de pesquisa e operação autônoma de swing trading baseada em IA. A IA realiza análise e decisão; dados, risco, execução, persistência e auditoria permanecem controlados por componentes determinísticos.



Princípio central: o LLM nunca envia ordens diretamente nem recebe credenciais. Ele produz uma decisão estruturada; o Risk Engine valida; o Execution Engine executa.

3. Decisões V1 Congeladas

Decisão	V1
Mercado	Criptomoedas
Instrumento	Spot
Direção	Long only
Alavancagem	0x
Capital virtual	R\$ 100,00
Exchange de referência	Binance
Execução	Paper Broker próprio + Demo/Test
Universo	10–20 ativos líquidos
Timeframes	1D / 4H / 1H
Trigger	Fechamento do candle 1H
IA	Analyst → Critic → Decision Maker
Decisão	LONG / HOLD / CLOSE
Risco por operação	1%
Posições simultâneas	1
Stop Loss	Obrigatório
Take Profit	Sim

Decisão	V1
Position sizing	Determinístico
Circuit breaker	10% de drawdown
News/Sentiment	Modulares
Auto-learning	Não na V1
Autoalteração de prompts/regras	Não
Backtest	Sim
Walk-forward	Sim
Replay	Sim
Paper trading	Sim
Exchange Demo	Sim
Custos	Fee + spread + slippage
Persistência	PostgreSQL
Cache/filas/locks	Redis
Comunicação	REST + WebSocket
Deployment	Docker
Dashboard	Sim
Auditoria	Event log + audit trail
Versionamento	Modelo + prompt + dados

Atualizações em tempo real utilizarão WebSocket.

8. Infraestrutura Docker

```
docker-compose
■■■ frontend
■■■ backend
■■■ worker
■■■ postgres
■■■ redis

Oracle Server
■■■ Docker
■■■ aplicação completa
```

Secrets ficam no backend/secret store e nunca no frontend ou dentro do prompt do LLM.

9. Roadmap de Implementação — 20 Fases

Fase	Componente	Escopo
01	Fundação	Docker Compose, frontend, backend, worker, PostgreSQL, Redis e healthcheck. Sem trading.
02	Banco de Dados	Schemas para configurações, decisões, ordens, fills, posições, trades, eventos e auditoria.
03	Configuração e Secrets	Variáveis de ambiente, secrets, configuração central e validação.
04	Backend Base	FastAPI, serviços, schemas, tratamento de erros e contratos iniciais.
05	Frontend Base	Shell visual, layout, navegação e componentes base.
06	WebSocket	Eventos em tempo real entre backend e frontend.
07	Market Data Engine	Coleta, normalização, candles 1H/4H/1D e histórico.
08	Universe Scanner	Liquidez, volume, spread e seleção de candidatos.
09	Paper Broker	Ordens, fills, taxas, spread, slippage e estados.
10	Execution Engine	Envio, cancelamento, reconciliação e proteção de ordens.
11	Portfolio Engine	Saldo, posições, custo, P&L; realizado/não realizado e equity.
12	Risk Engine	Risco de 1%, position sizing, limites, stop obrigatório e circuit breaker.
13	AI Gateway	Providers, modelos, retries, timeouts, structured output e versionamento.
14	AI Agents	Analyst, Critic e Decision Maker.
15	Decision Engine	Orquestração Market State → IA → Trade Intent → Risk.
16	Replay Engine	Reprodução temporal sem look-ahead.
17	Backtester	Backtest, custos, walk-forward e métricas.
18	Dashboard	Operações, posições, performance, IA, configurações e experimentos.
19	Auditoria e Logs	Eventos, auditoria, rastreabilidade e reconciliação.
20	Integração Final	Integração ponta a ponta, testes, hardening, smoke tests e aprovação V1.

10. FASE 01 — Fundação do Projeto

Objetivo: criar somente a fundação executável, sem IA, exchange ou lógica de trading.

Visão arquitetural

```
docker-compose
■■■ frontend
■■■ backend
■■■ worker
■■■ postgres
■■■ redis
```

Entregáveis: backend com api/services/models/schemas/core; React + TypeScript; worker; PostgreSQL e Redis persistentes; GET /health retornando status ok; Dashboard com “System Online”.

Prompt de Execução

```
Você é um engenheiro de software sênior.

Implemente EXATAMENTE a fundação do projeto descrita abaixo.

NÃO crie:
- lógica de trading;
- IA;
- exchange;
- regras financeiras;
- funcionalidades adicionais.

TECNOLOGIAS:

Frontend:
- React
- TypeScript

Backend:
- Python
- FastAPI

Infraestrutura:
- Docker Compose
- PostgreSQL
- Redis

CONTAINERS:

- frontend
- backend
- worker
- postgres
- redis

BACKEND:

Criar:

app/
api/
services/
models/
schemas/
core/

Criar endpoint:

GET /health

Resposta:

{
  "status": "ok"
}

FRONTEND:
```

Criar página:

Dashboard

Conteúdo:

System Online

WORKER:

Criar container preparado para futuras tarefas.

POSTGRES:

Criar volume persistente.

REDIS:

Criar volume persistente.

ENTREGAR:

- estrutura completa;
- código;
- Docker Compose;
- instruções de execução;
- árvore de arquivos.

Não implementar nada fora deste escopo.

Prompt de Auditoria

Você é um auditor de arquitetura.

Analise TODO o código produzido.

Verifique:

1. Existe docker-compose.
2. Existem os containers:
 - frontend
 - backend
 - worker
 - postgres
 - redis
3. O backend possui:
 - api
 - services
 - models
 - schemas
 - core
4. Existe:

GET /health
5. A resposta é:

```
{
"status": "ok"
}
```
6. Existe Dashboard no frontend.
7. O Dashboard apresenta:

System Online
8. O worker existe.
9. PostgreSQL possui persistência.
10. Redis possui persistência.

11. NÃO existe:

- lógica de trading;
- IA;
- integração com exchange;
- regras financeiras;
- execução de ordens.

12. Não foram implementadas funcionalidades pertencentes às fases futuras.

13. Todos os containers conseguem iniciar.

14. O código respeita a arquitetura definida.

IMPORTANTE:

Não considere a intenção do desenvolvedor.
Verifique o código efetivamente implementado.

Se qualquer requisito estiver ausente ou diferente,
marque como REPROVADO.

Resposta final obrigatória:

APROVADO

ou

REPROVADO

Depois liste todas as evidências encontradas.

Critério de aprovação: todos os containers sobem, /health funciona, Dashboard abre e nenhuma funcionalidade fora do escopo foi introduzida.

11. Contrato Obrigatório das Fases 02–20

Toda fase deverá possuir obrigatoriamente: Objetivo → Escopo → Visão arquitetural → Entregáveis → Prompt de implementação → Prompt de auditoria → Critérios de aprovação.

O agente programador não deve decidir arquitetura. Diante de uma ambiguidade relevante, deve reportá-la ao arquiteto em vez de inventar uma solução.

GOVERNANÇA

1. Ler o Blueprint mestre.
2. Implementar somente a fase autorizada.
3. Não alterar contratos anteriores sem autorização.
4. Não antecipar funcionalidades de fases futuras.
5. Criar testes da própria fase.
6. Entregar evidências de execução.
7. Rodar a auditoria.
8. Corrigir somente divergências identificadas.
9. Registrar aprovação antes de avançar para a próxima fase.

12. Versionamento e Evidências

Cada fase aprovada deve registrar: número da fase, commit, timestamp, resumo, testes executados, resultado da auditoria, divergências corrigidas e status APPROVED/REJECTED.

Recomenda-se manter um artefato por fase, por exemplo:

```
PHASE_01_APPROVED.md
```

Prompts, modelos, configurações experimentais e datasets utilizados também devem ser versionados.

13. Gates de Validação da V1

```
GATE 1 – SOFTWARE
Sem erros críticos e containers estáveis.

GATE 2 – DATA
Sem look-ahead e dados temporalmente consistentes.

GATE 3 – EXECUTION
Ordens reconciliadas e estados consistentes.

GATE 4 – BACKTEST
Resultados reproduzíveis com custos realistas.

GATE 5 – REPLAY
Comportamento reproduzível sem acesso ao futuro.

GATE 6 – PAPER
Volume suficiente de decisões para análise inicial.

GATE 7 – DEMO
Integração com exchange de teste validada.

GATE 8 – END-TO-END
Fluxo completo funcionando e auditável.
```

14. Estado do Artefato

Versão: V1.0 — Blueprint Mestre

Conteúdo: conceitual + arquitetural + planejamento de implementação.

Próximo passo: especificação técnica detalhada da fase autorizada, seguida de implementação e auditoria.

Regra: nenhuma fase é concluída sem auditoria e aprovação.